Laboratorio 1

Santiago Blanco
Felipe Paladino
Piero Saucedo

22 de abril de 2026

Introducción

En el presente informe se documenta el desarrollo del Laboratorio 1 de la asignatura IoT y Sistemas Embebidos, centrado en la introducción al uso de la placa `ESP32-S2-KALUGA-1` y del entorno de desarrollo ESP-IDF.
A lo largo de la labor se describen los principales pasos realizados para la configuración del entorno, la creación y compilación del proyecto, el análisis de archivos generados por el sistema de *build*, el proceso de flasheo y monitoreo de la placa, así como la implementación de una biblioteca basada en el control del LED RGB integrado.
Se incluyen las respuestas a las preguntas planteadas en el laboratorio, junto con una breve explicación técnica de cada procedimiento realizado.

Primera Parte

Ejercicio 1

¿Qué secciones tienen los archivos generados? ¿Cuál es el propósito de cada uno?
Al crear un proyecto nuevo con ESP-IDF, se genera automáticamente una estructura base de directorios y archivos como la siguiente:

```
|-- CMakeLists.txt
|-- main
|   |-- CMakeLists.txt
|   `-- laboratorio.c
```

Cada archivo dentro del proyecto cumple un rol específico dentro del sistema de compilación basado en CMake y el modelo modular de ESP-IDF.

Archivo `CMakeLists.txt` (raíz del proyecto). Este archivo corresponde a la configuración principal del proyecto. Su función es definir la estructura general del *build system* y permitir que CMake pueda interpretar correctamente el proyecto. En particular:

- Define la versión mínima de CMake requerida.
- Incluye el archivo `project.cmake` provisto por ESP-IDF.
- Declara el nombre del proyecto mediante `project(...)`.

El contenido típico del archivo es:

```
cmake_minimum_required(VERSION 3.16)

include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(laboratorio)
```

Carpeta `main`. ESP-IDF utiliza por convención la carpeta `main` como el directorio principal donde se almacena el código fuente de la aplicación. Dentro de esta carpeta se ubican los archivos que componen el programa principal, junto con un archivo adicional de configuración para CMake.

Archivo `laboratorio.c`. Este archivo contiene la lógica principal del programa desarrollado. En ESP-IDF es obligatorio que dentro del código exista la función:

```
void app_main(void)
```

La misma cumple el rol de punto de entrada de la aplicación, equivalente a la función `main()` en un programa convencional en C.

Archivo `CMakeLists.txt` (dentro de `main`). Este archivo define el directorio `main` como un componente del proyecto dentro del sistema modular de ESP-IDF. Su objetivo es indicar qué archivos fuente deben compilarse y qué directorios deben considerarse como rutas de inclusión para los headers.
Su contenido típico es:

```
idf_component_register(SRCS "laboratorio.c"
                       INCLUDE_DIRS ".")
```

¿Qué cambios ocurren en el workspace luego de compilar? ¿Qué información provee el compilador?
Luego de ejecutar el comando:

```
idf.py build
```

el entorno genera automáticamente una carpeta `build`, donde se almacenan los archivos intermedios y finales del proceso de compilación.
De esta forma, la estructura del proyecto pasa a ser:

```
|-- build
|-- CMakeLists.txt
|-- main
|   |-- CMakeLists.txt
|   `-- laboratorio.c
|-- README.md
`-- sdkconfig
```

Para empezar, olvídese del uso de VSCode. La compilación se realiza correctamente mediante la herramienta `idf.py` provista por ESP-IDF.
Dentro de la carpeta `build` se generan archivos relevantes como:

- binarios finales (`.bin`)
- ejecutables (`.elf`)
- archivos objeto (`.obj`)

- mapa de memoria del linker (`.map`)
- archivos de soporte para flasheo (`.json`)

Durante la compilación, el compilador y el sistema de build proporcionan información relevante, incluyendo errores, advertencias y el progreso de construcción de cada componente.

Ejercicio 2

¿Qué opciones relevantes encontrás en el sdkconfig? ¿Qué diferencias aparecen entre sdkconfig y sdkconfig.old luego de guardar un cambio?
El archivo `sdkconfig` contiene parámetros de configuración del proyecto en formato `clave=valor`. Estos determinan el comportamiento del sistema durante la compilación y ejecución.
Aunque puede editarse manualmente, se recomienda modificarlo mediante la herramienta:

```
idf.py menuconfig
```

ya que permite asegurar consistencia entre configuraciones.
Algunos parámetros relevantes que pueden encontrarse en `sdkconfig` incluyen:

- `CONFIG_LOG_DEFAULT_LEVEL` (nivel de logging).
- `CONFIG_FREERTOS_HZ` (frecuencia del tick de FreeRTOS).
- `CONFIG_ESPTOOLPY_FLASHMODE` (modo de flash).
- `CONFIG_ESPTOOLPY_FLASHFREQ` (frecuencia de flash).
- `CONFIG_ESPTOOLPY_FLASHSIZE` (tamaño de flash).

Respecto a la diferencia entre `sdkconfig` y `sdkconfig.old`, se observa que `sdkconfig.old` corresponde a una copia del estado anterior del archivo. Por lo tanto, luego de guardar cambios mediante `menuconfig`, es esperable que existan diferencias entre ambos.

Ejercicio 3

¿Qué información contienen esos archivos? ¿Para qué sirven en el proceso de flasheo?
Se agregaron al archivo principal las siguientes variables globales:

```
#define ARRAY_SIZE  12

int exampleData;
char exampleArray[ARRAY_SIZE];
```

Luego de compilar el proyecto, dentro de la carpeta `build/` se generaron archivos relevantes para el proceso de flasheo.

Archivo `flasher_args.json`. Este archivo contiene la información necesaria para flashear el dispositivo, incluyendo:

- parámetros de la memoria flash (modo, tamaño y frecuencia)
- direcciones donde escribir cada binario generado

En particular, especifica en qué direcciones se escriben el bootloader, la tabla de particiones y la aplicación principal.

Archivo `project_description.json`. Este archivo contiene información general del proyecto compilado, como:

- nombre del proyecto
- arquitectura objetivo
- compilador utilizado
- componentes compilados
- referencias a archivos generados (`.elf` y `.bin`)

¿Qué tamaño ocupan en memoria y en qué direcciones están alojadas?
Las variables `exampleArray` y `exampleData` se ubican en la sección `.bss`, ya que son variables globales no inicializadas.

- `exampleArray`: ocupa `0xc` bytes (12 bytes).
- `exampleData`: ocupa `0x4` bytes (4 bytes).

Ambas se alojan dentro de la región de memoria correspondiente a `.dram0.bss`, que agrupa las variables globales no inicializadas en RAM.

¿Cómo cambia el tamaño y la dirección de `exampleArray` en el mapa de memoria?
Al modificar el valor de `ARRAY_SIZE` de 12 a 10, el tamaño de `exampleArray` cambia de `0xc` (12 bytes) a `0xa` (10 bytes).
Este cambio puede afectar también la dirección final de la variable, ya que el linker reorganiza los símbolos dentro de la sección `.dram0.bss` según el tamaño y orden de las variables globales.

Ejercicio 4

Antes de realizar el proceso de flasheo fue importante verificar que el entorno estuviera correctamente configurado y que la versión de ESP-IDF utilizada fuera la adecuada. En particular, se trabajó con la versión **ESP-IDF v5.5**, ya que versiones diferentes pueden generar incompatibilidades con la placa utilizada.
El primer paso consistió en comprobar que el sistema operativo reconociera correctamente la placa conectada por USB. En Linux esto se verificó mediante:

```
ls /dev/ttyUSB*
```

Luego fue necesario cargar manualmente el entorno de trabajo de ESP-IDF:

```
. ~/esp/esp-idf/export.sh
```

Además, fue necesario seleccionar explícitamente el microcontrolador objetivo del proyecto:

```
idf.py set-target esp32s2
```

Posteriormente, se flasheó el firmware utilizando:

```
idf.py -p /dev/ttyUSB0 flash monitor
```

Durante el flasheo, la terminal indicó el borrado de sectores de la memoria flash, la escritura de los binarios correspondientes y la verificación final. Se transfirieron tres secciones principales:

- `bootloader` (código de arranque inicial),
- `partition-table` (organización de la memoria flash),
- `app` (firmware principal).

Una vez finalizado, el monitor serie mostró mensajes de arranque provenientes del bootloader y del sistema de logging interno de ESP-IDF.

¿Qué imprime el programa por defecto al arrancar? ¿De dónde proviene ese mensaje?
Al iniciar la placa, el monitor serie imprime información del sistema, incluyendo el tipo de reset, tabla de particiones y carga de la aplicación. Dichos mensajes provienen del bootloader y del framework ESP-IDF, y no del código escrito por el usuario.

Agregar un `printf` propio y verificar en el monitor.
Se agregó un mensaje propio mediante `printf` dentro de `app_main()`. Luego de recompilar y flashear nuevamente el proyecto, se verificó en el monitor serie que el mensaje aparece correctamente, confirmando la salida por UART.

Segunda Parte

Ejercicio 1

El LED RGB WS2812C utiliza un protocolo de comunicación de un solo pin (*single-wire*), basado en codificación por tiempos (NRZ). El LED recibe la información mediante el pin `DIN`.
El protocolo se basa en el envío de tramas de 24 bits correspondientes a un color en formato **G-R-B**, donde cada componente ocupa 8 bits.

- Primeros 8 bits (`G7:G0`): componente verde.
- Siguientes 8 bits (`R7:R0`): componente rojo.
- Últimos 8 bits (`B7:B0`): componente azul.

Cada bit se representa mediante tiempos críticos:

1. **T0H y T0L**: codificación de un bit `0`.
2. **T1H y T1L**: codificación de un bit `1`.
3. **RESET**: pulso bajo prolongado al final de la transmisión.

Pseudocódigo de control del WS2812C

```
function send_bit(bit):
    if bit == 0:
        set_pin_high()
        wait(T0H)
        set_pin_low()
        wait(T0L)
```

```
    else:
        set_pin_high()
        wait(T1H)
        set_pin_low()
        wait(T1L)

function send_color(G, R, B):
    data = concatenate(G, R, B)
    for each bit in data (MSB first):
        send_bit(bit)
    set_pin_low()
    wait(RESET_TIME)
```

Ejercicio 2

Para simplificar el control del LED RGB embebido en la placa, se implementó una librería propia dentro de la carpeta `components/rgb_led/`. Esta librería permite abstraer el uso del driver provisto por ESP-IDF y trabajar de forma más clara desde el archivo principal.

Se definió una estructura llamada `color_t` que representa una tupla de colores en formato RGB:

```
typedef struct color_s {
    uint8_t r;
    uint8_t g;
    uint8_t b;
} color_t;
```

Además, se implementó una función encargada de establecer el color del LED utilizando el driver `led_strip`:

```
esp_err_t led_set_color(led_strip_t *led, color_t colors)
```

Esta función configura el color del píxel mediante `set_pixel()` y actualiza el LED físicamente mediante `refresh()`.

Ejercicio 3

Se implementó una librería de delay basada en la función nativa:

```
esp_rom_delay_us(uint32_t number_of_us);
```

A partir de esta función se crearon dos funciones *wrapper* para generar delays en milisegundos y en segundos, exponiendo:

```
void delay_ms(uint32_t ms);
void delay_s(uint32_t s);
```

Dificultades respecto a Watchdog

Durante las pruebas, el equipo se encontró con reinicios inesperados debido a la activación del Task Watchdog Timer (TWDT), ya que la función de delay bloqueaba la ejecución por demasiado tiempo sin ceder control al scheduler. Para solucionar este problema, se modificó la configuración desde `idf.py menuconfig`. Si bien deshabilitar el Watchdog puede ser útil en un contexto de laboratorio, en sistemas reales se recomienda utilizar mecanismos como `vTaskDelay()` para evitar bloqueos prolongados.

Ejercicio 4

¿Por qué se usa el canal RMT del ESP32-S2 en lugar de generar la señal por software con un GPIO y la función de delay? ¿Qué ventajas ofrece el hardware en este caso?

El protocolo del WS2812C requiere tiempos extremadamente precisos para codificar los bits, por lo cual implementar la señal por software mediante delays es poco confiable, especialmente en un sistema con interrupciones y multitarea. El canal RMT permite generar estas señales con precisión por hardware, asegurando estabilidad temporal, reduciendo carga de CPU y evitando errores en la transmisión debido al scheduler o interrupciones del sistema.

Conclusión

El desarrollo de este laboratorio permitió establecer los fundamentos para trabajar con el ecosistema ESP-IDF y la placa ESP32-S2-KALUGA-1. A través del análisis del *build system*, se logró comprender el funcionamiento general de CMake, la estructura modular de componentes y la importancia del archivo `sdkconfig` en la configuración del sistema.

Se estudió además el proceso de compilación y flasheo, verificando la relevancia de archivos como `flasher_args.json` y `project_description.json`, los cuales contienen información esencial para transferir correctamente el firmware a la memoria flash.

Por otra parte, se investigó el protocolo del LED WS2812C y se implementaron librerías propias para controlar el LED RGB y generar delays, introduciendo al equipo a prácticas básicas de modularización en sistemas embebidos.

Anexo: Logs de error del sistema

```
E (5264) task_wdt: Task watchdog got triggered.
The following tasks/users did not reset the watchdog in time:
E (5264) task_wdt:  - IDLE (CPU 0)
E (5264) task_wdt: Tasks currently running:
E (5264) task_wdt: CPU 0: main
E (5264) task_wdt: Print CPU 0 (current core) backtrace

Backtrace: 0x400887F7:0x3FFC1B50 0x40088B88:0x3FFC1B70 0x4002652D:0x3FFC1BA0 0x4001AA90:0x3FFC4C40
     0x4000D898:0x3FFC4C50 0x40086F42:0x3FFC4C70 0x40086EC8:0x3FFC4C90 0x40095539:0x3FFC4CC0 0
    x4002A785:0x3FFC4CF0
--- 0x400887f7: task_wdt_timeout_handling at /home/jwillmore/Documents/esp32/esp-idf/components/
    esp_system/task_wdt/task_wdt.c:430
--- 0x40088b88: task_wdt_isr at /home/jwillmore/Documents/esp32/esp-idf/components/esp_system/
    task_wdt/task_wdt.c:503
--- 0x4002652d: _xt_lowint1 at /home/jwillmore/Documents/esp32/esp-idf/components/esp_system/
    task_wdt/task_wdt.c:503
--- 0x4001aa90: xthal_get_ccount in ROM
--- 0x4000d898: ets_delay_us in ROM
--- 0x40086f42: delay_ms at /home/jwillmore/Documents/esp32/esp-idf/projects/laboratorio/
    components/delay/delay.c:14
--- 0x40086ec8: app_main at /home/jwillmore/Documents/esp32/esp-idf/projects/laboratorio/main/
    laboratorio.c:39
--- 0x40095539: main_task at /home/jwillmore/Documents/esp32/esp-idf/components/freertos/
    app_startup.c:208
--- 0x4002a785: vPortTaskWrapper at /home/jwillmore/Documents/esp32/esp-idf/components/freertos/
    FreeRTOS-Kernel/portable/xtensa/port.c:139
```

Listing 1: Error de Task Watchdog en ESP32-S2